

Module:2 Data Representation and Computer Arithmetic

- Algorithms for fixed point arithmetic operations:
 - Multiplication (Booths, Modified Booths),
 - Division (restoring and non-restoring) –
- Algorithms for floating point arithmetic operations -
- Representation of nonnumeric data (character codes).

Booth's Algorithm

- Booth's multiplication algorithm defines
 - a multiplication algorithm that can multiply
 - **Two signed binary numbers in two's complement.**
- An accumulator is a type of register
 - included in a CPU (Central Processing Unit).
 - It acts as a temporary storage location
 - which holds an intermediate value in mathematical and logical calculations.

Notations

- **Multiplicand (M)**
- **Multiplier (Q)**
- **Accumulator (A)** initialized to 0
- **Q_{-1}** initialized to 0
 - single-bit register used in Booth's multiplication algorithm
 - to store the previous least significant bit (LSB) of the multiplier Q.
- **n** = number of bits use to represent M, Q, and A

A = 0 (n bits)

Q = Multiplier (n bits)

M = Multiplicand (n bits)

$Q_{-1} = 0$

Counter = n

Booth's algorithm examines **two bits** in each step:

- The **current LSB of Q (Q_0)**
- And the **previous value of Q_0** , which is Q_{-1}

This pair determines whether to **add**, **subtract**, or do **nothing** based on the following rule:

Q_0	Q_{-1}	Action
0	0	No Arithmetic Operation
1	1	No arithmetic Operation
1	0	Subtract M from A ($A-M$)
0	1	Add M to A ($A+M$)

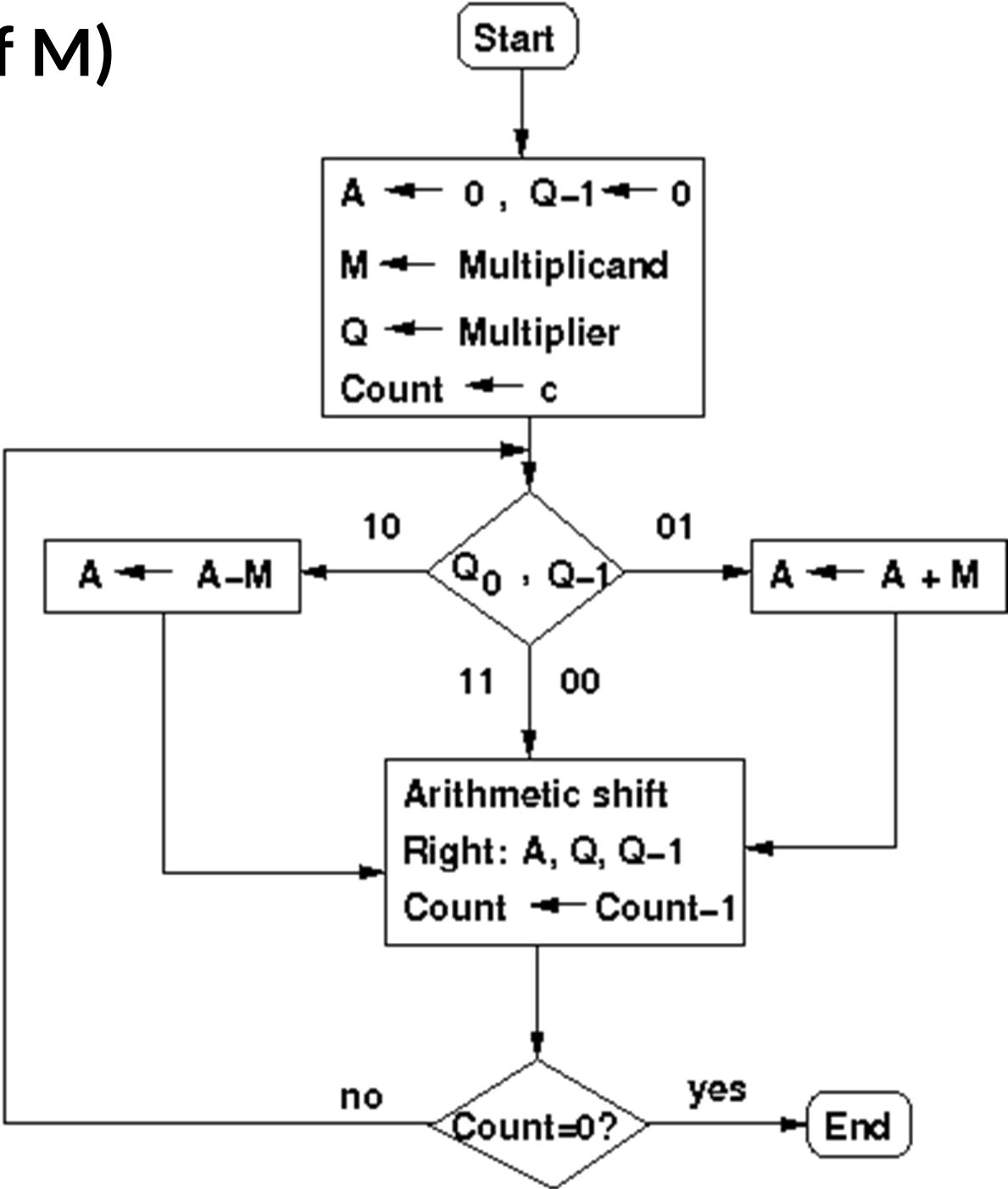
Multiply 7x3 using Booth's Algorithm

Inputs:

- **Multiplicand (M)** = 0111 (7)
- **Multiplier (Q)** = 0011 (3)
- **Initial A** = 0000
- **Initial Q_{-1}** = 0
- **n = 4**(minimum 4 bits are required to represent in sign form)

M=0111 , -M=1001(2's complement of M)

A	Q	Q ₁ ⁻	Q ₀	Count	Operation
0000	0011	0	1	4	A = A - M → 1001
1001	0011	0	1	4	Arithmetic Right Shift
1100	1001	1	1	3	No arithmetic Operation
1100	1001	1	1	3	Arithmetic Right Shift
1110	0100	1	0	2	A = A + M → 0101
0101	0100	1	0	2	Arithmetic Right Shift
0010	1010	0	0	1	No arithmetic Operation
0010	1010	0	0	1	Arithmetic Right Shift
0001	0101	0	1	0	END



- The final result is in AQ.
 - If MSB of AQ is 0
 - i.e result is positive .
 - Convert the AQ in decimal form
 - Example
 - AQ= 00010101 MSB =0 i.e answer is positive find its decimal equivalent =21
 - If the MSB of AQ is 1
 - i.e result is negative.
 - Find its 2's complement and add minus sign.

Example

- Perform using booths algorithm
- 6×2
- 7×-3
- -7×3

Modified booths algo

- Multiplicand=10
- 1. Choose the multiplier
 - 12=001100=multiplier
- 2. Calculate the booth recoded value of multiplier
 - 0 +1 0 -1 0 0

0	0	0
1	0	+1
0	1	-1
1	1	0

3. Calculate the bit pair recoded multiplier value

- $\begin{bmatrix} 0 & +1 \\ +1 & \end{bmatrix} \begin{bmatrix} 0 & -1 \\ -1 & \end{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & \end{bmatrix}$

4. Verify its equivalent value and compare with multiplier value

- $12 = 001100$
- $+1 -1 0 = 16 + -4 + 0$

5. Multiply the original multiplicand with recoded multiplier

- $001010 \times +1 -1 0$

Division: Restoring method for unsigned integer

- Perform division of 11 by 3
 - Answer : **11 dividend, 3 Divisor, quotient 3, remainder 2**
 - Restoring method
 - Non restoring method

Restoring method

Register Setup

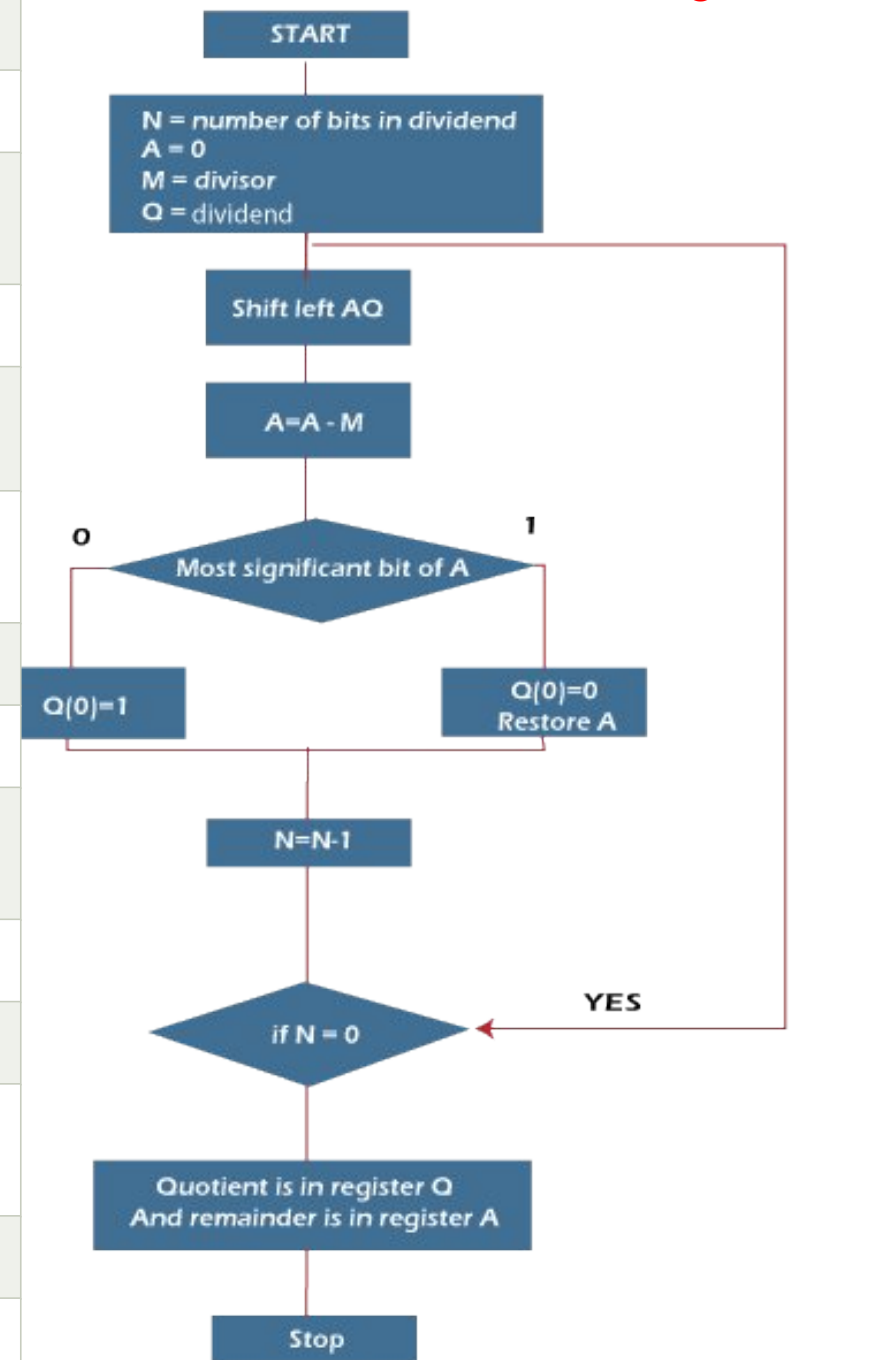
We will use the following Input :

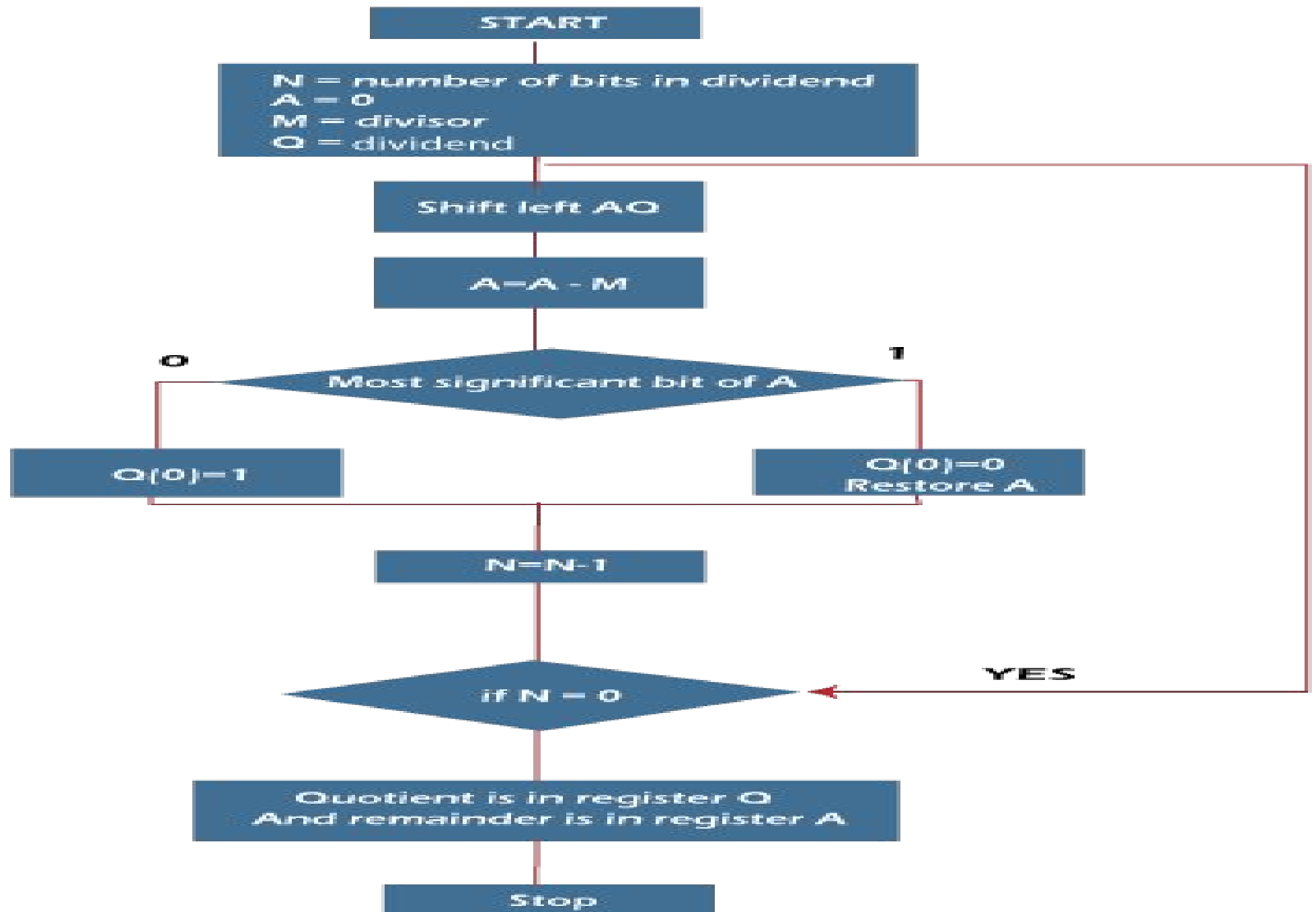
- **A (Accumulator)** – initially 00000
- **Q (Dividend)** – 01011 (5 bits for shifting)
- **M (Divisor)** – 00011
- **n (number of bits)** – 5 (for Q's bits)

- **Output:**
 - Quotient is in Register Q
 - Remainder is in register A

N	M (bits)	(N+1) A (bits)	(N+1) Q (bits)	Operation
4	00011	00000	1011	Initialize
	00011	00001	011_	Shift left AQ
	00011	1 1110	011_	$A = A - M = A + M' + 1$
	00011	00001	0110	$Q[0] = 0$ And restore A
3	00011	00010	110_	Shift left AQ
	00011	1 1111	110_	$A = A - M$
	00011	00010	1100	$Q[0] = 0$ AND restore A
2	00011	00101	100_	Shift left AQ
	00011	0 0010	100_	$A = A - M$
	00011	00010	1001	$Q[0] = 1$
1	00011	00101	001_	Shift left AQ
	00011	00010	001_	$A = A - M$
	00011	00010	0011	$Q[0] = 1$

Restoring method





Algorithm

Input: Dividend Q, Divisor M (both in binary)

Output: Quotient (in Q), Remainder (in A)

1. Initialize:

- a. $A \leftarrow 0$ (n+1 bits)
- b. $Q \leftarrow \text{Dividend}$ (n bits)
- c. $M \leftarrow \text{Divisor}$ (n+1)
- d. $n \leftarrow \text{number of bits in } Q \text{ (i.e., loop counter)}$

2. Repeat n times until $n \neq 0$:

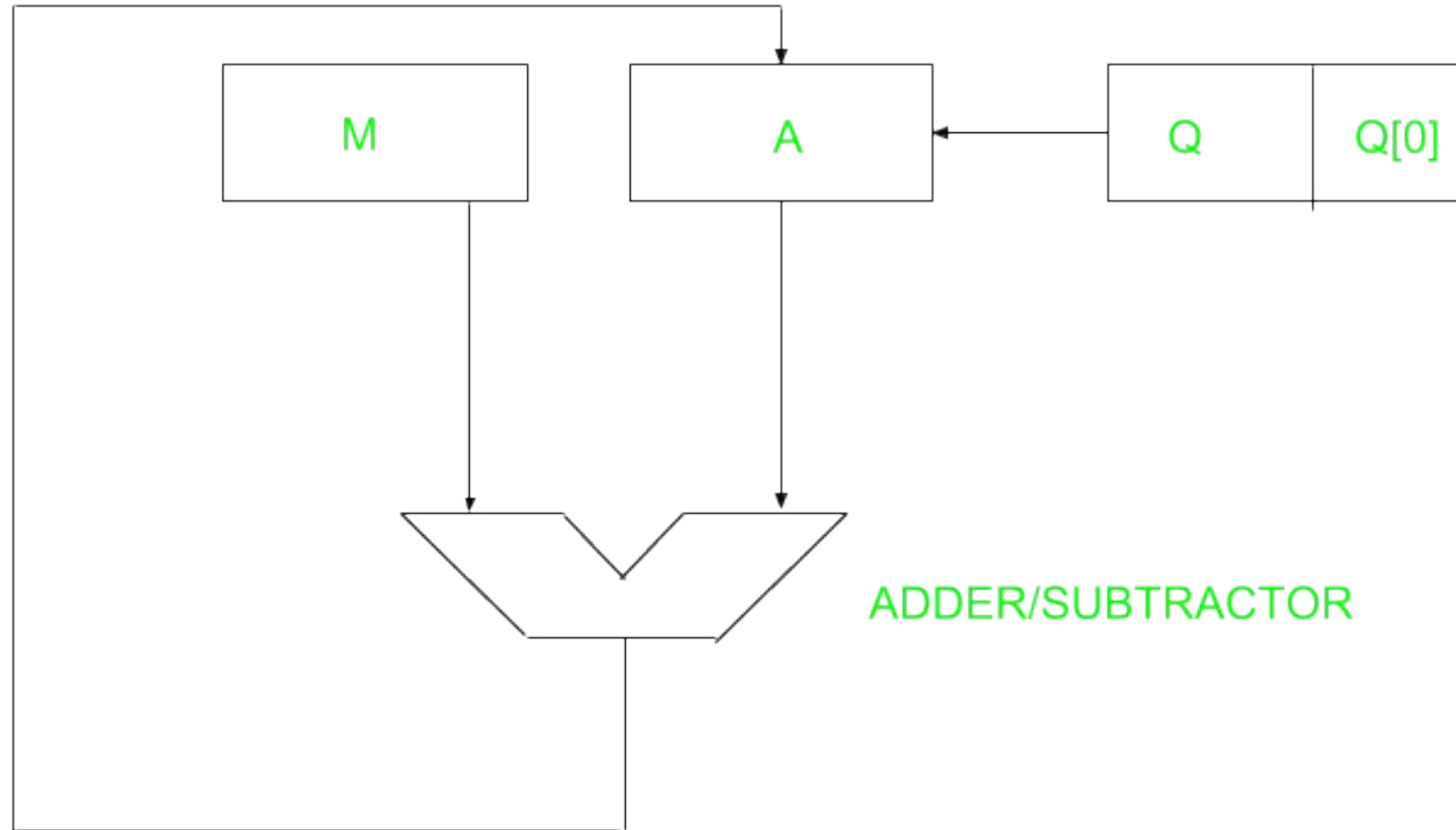
- a. Shift left A and Q ($A, Q \leftarrow A, Q \ll 1$)
- b. $A \leftarrow A - M$
- c. If $A < 0$ i.e MSB is 1 then
 - i. **Restore A**
 - ii. $Q_0 \leftarrow 0$
- else
 - $Q_0 \leftarrow 1$
- d. Decrement $n \leftarrow n - 1$

3. Result:

Quotient $\leftarrow Q$

Remainder $\leftarrow A$

Hardware



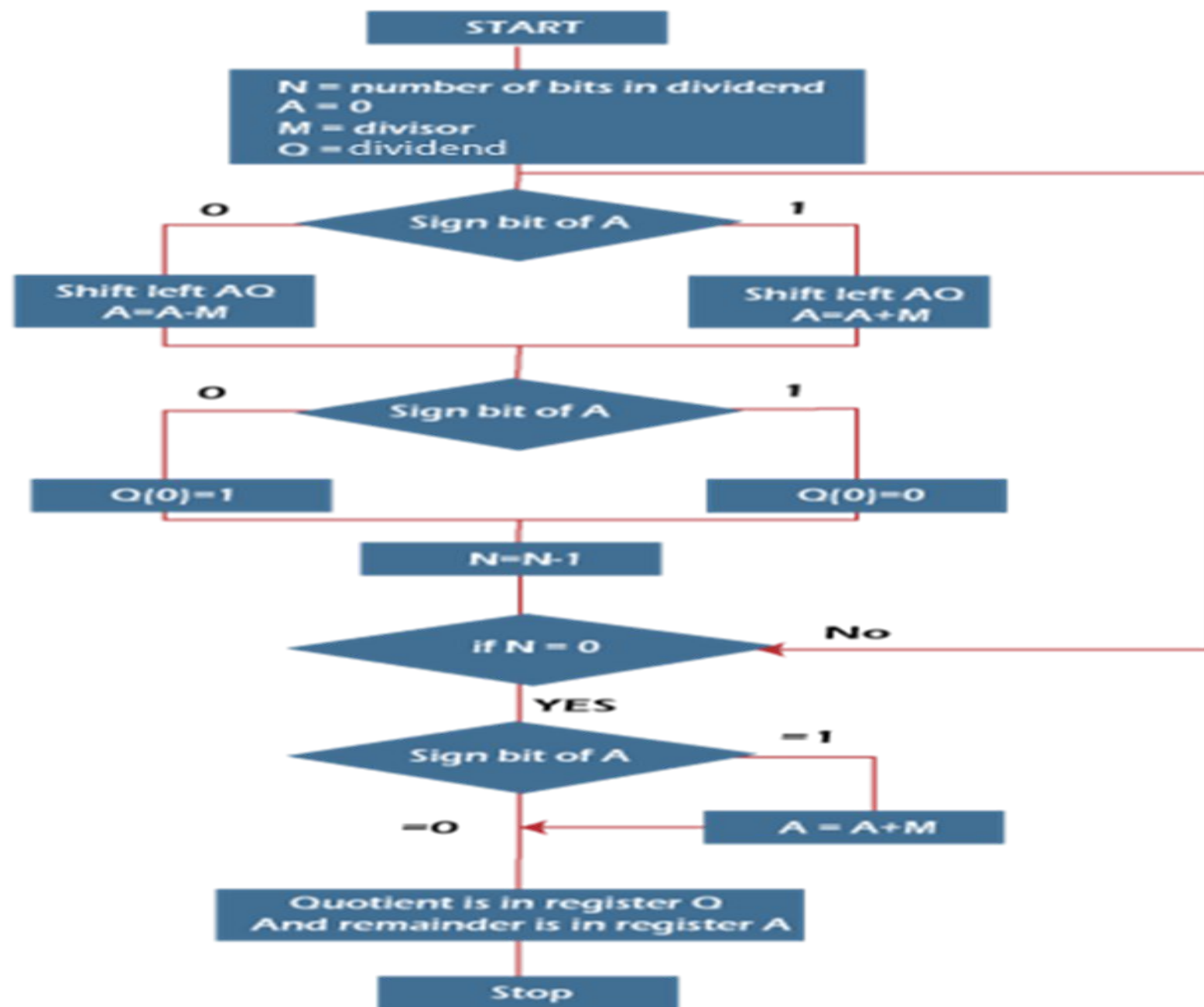
Non restoring method

perform a Non-Restoring Division algorithm with the help of an Unsigned integer.

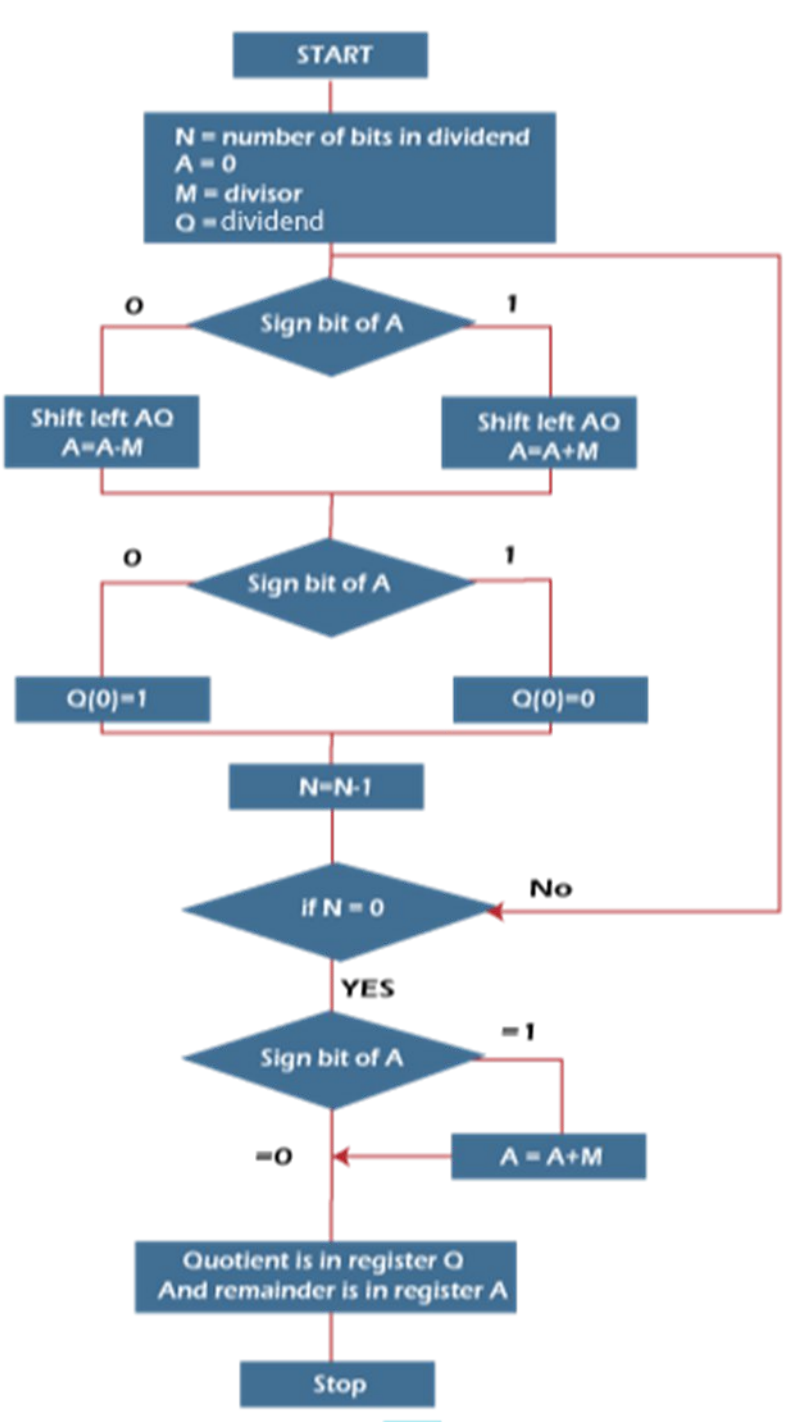
Dividend = 11

Divisor = 3

-M = 11101



N	M	A	Q	Action
4	00011	00000	1011	Begin
	00011	00001	011_	Shift left AQ
	00011	11110	011_	$A = A - M$
3	00011	11110	0110	$Q[0] = 0$
	00011	11100	110_	Shift left AQ
	00011	11111	110_	$A = A + M$
2	00011	11111	1100	$Q[0] = 0$
	00011	11111	100_	Shift left AQ
	00011	00010	100_	$A = A + M$
1	00011	00010	1001	$Q[0] = 1$
	00011	00101	001_	Shift left AQ
	00011	00010	001_	$A = A - M$
0	00011	00010	0011	$Q[0] = 1$



Fixed point and Floating Point

- Based on the number system **two basic data types** are implemented in the computer system.
 - **Fixed point numbers**
 - **Floating point numbers**

Floating point Arithmetic: Representation

- We can represent a number in the form

$$\text{sign } S \times B^{\text{sign } E}$$

- This number can be stored in a binary word with three fields:
 - Sign: plus or minus (+/-)
 - Significand/mantisa S
 - Exponent E

Floating point arithmetic-Types

Type	Description	Example Formats
Single Precision	• 32-bit format with ~7 decimal digits of accuracy	IEEE 754 float32
Double Precision	• 64-bit format with ~15–17 decimal digits of accuracy	IEEE 754 float64
Extended Precision	• Higher than double, implementation-dependent (e.g., 80-bit, 96-bit, 128-bit)	float80, float128
Half Precision	• 16-bit format with ~3–4 decimal digits of accuracy	IEEE 754 float16
Quadruple Precision	• 128-bit format, high accuracy (~34 decimal digits)	IEEE 754 float128
Bfloat16 (Brain Float)	• 16-bit with 8-bit exponent, used in ML for range similar to float32	Google TPUs, PyTorch
Custom Precision	• User-defined precision for specific hardware or software applications	FPGA/GPU configurations

Precision

- How many significant digits a floating-point number can accurately represent.
- In binary floating-point (IEEE 754),
 - precision depends on the **number of bits in the mantissa** (also called the **fraction** or **significand**).
- **Formula to Approximate Decimal Precision:**
$$\text{Decimal Digits of Precision} \approx \text{Mantissa Bits} \times \log_{10}(2)$$
$$\approx \text{Mantissa Bits} \times 0.301$$

Precision: Importance

- **Scientific calculations**

→ Prevents rounding errors and maintains numerical accuracy in simulations.

- **Financial computations**

→ Ensures exact decimal values, avoids losses due to rounding.

- **Graphics / Games**

→ Precision may affect rendering quality and visual artifacts.

- **Machine Learning**

→ Trade-off between speed (float16/float32) and accuracy (float64).

IEEE754 Precision Table

Format	Mantissa Bits	Approx. Decimal Precision	Total Bits
Half (float16)	10	~3 to 4 digits	16
Single (float32)	23	~6 to 7 digits	32
Double (float64)	52	~15 to 17 digits	64
Quad (float128)	112	~33 to 36 digits	128

Example: Value of Pie

Precision Type	Format (bits)	Stored Value of π	Approx. Decimal Digits	Accurate?
Half	16-bit (float16)	3.1406	~3–4 digits	Low
Single	32-bit (float32)	3.1415927	~6–7 digits	Rounded
Double	64-bit (float64)	3.141592653589793	~15–17 digits	Accurate
Quadruple	128-bit (float128)	3.1415926535897932 384626433832795	~34 digits	Very precise

Structure of IEEE 754 Floating Point Number

All IEEE 754 floating-point numbers have the form:

$$\text{Value} = (-1)^S \times 1.M \times 2^{E - \text{Bias}}$$

Where:

- **S: Sign bit** (1 = negative, 0 = positive)
- **M: Mantissa** (fraction bits)
- **E: Exponent** (biased)
- **Bias:** Used to allow both positive and negative exponents

32 Bit floating point Format(single Precision)

1 bit	8 bit	23 bit
Sign Bit	Biased Exponent	Significand/Mantissa

Example: IEEE 754 Binary (32-bit)

- **Binary**

- 0 10000001 100100000000000000000000

1 bit	8 bit	23 bit
Sign Bit	Biased Exponent	Significand/Mantisa
0	10000001	100100000000000000000000

- **Sign bit (S)** = 0
- **Exponent (E)** = 10000001 = 129
- **Mantissa (M)** = 100100000000000000000000

Calculation

Sign (S):

- $S = 0 \rightarrow$ Number is **positive**
- So, $(-1)^S = 1$

Exponent (E):

- Binary: 10000001 $\rightarrow$ Decimal: 129
- Bias for 32-bit (single precision) = **127**
- So, actual exponent:
 $E - \text{Bias} = 129 - 127 = 2$

Mantissa (M):

- Mantissa bits: 100100000000000000000000
- This represents the fraction part after the leading 1.

To convert to decimal:

$$1.M = 1 + 1/2^1 + 0/2^2 + 0/2^3 + 1/2^4 + \dots = 1 + 0.5 + 0 + 0 + 0.0625 = 1.5625$$

Final Calculation: $\text{Value} = (-1)^S \times 1.M \times 2^{E - \text{Bias}}$

$$\begin{aligned} \text{Value} &= 1 \times 1.5625 \times 2^2 = 1.5625 \times 4 \\ &= 6.25 \end{aligned}$$

The IEEE 754 value is: 6.25

Bias

- Computers store bits as **unsigned integers** (only positive).
- We need both **positive and negative exponents** in floating point math.
- **Bias** is a fixed value.
- It is added to the **actual exponent** to allow the **exponent field to represent both positive and negative exponents** using only **unsigned binary numbers**.

Bias

- To represent both **positive and negative exponents** using only unsigned binary values.
- Formula:
 - n is the number of bit in exponent

$$\text{Bias} = 2^{n-1} - 1$$

It puts **0 in the centre** of the available range(almost symetric):

$$\text{Bias} = (2^n / 2) - 1 = 2^{n-1} - 1$$

This gives:

- $2^{n-1} - 1$ values **below zero**
- 2^{n-1} values **above zero**

Bias

Term	Meaning
Stored Exponent	The value stored in the exponent bits (always a positive binary number)
Actual Exponent	The true exponent used in the formula: $1.M \times 2^{\text{actual exponent}}$

- IEEE 754 stores the **actual exponent** as:
 - **Stored Exponent=Actual Exponent+Bias**
- we want the **range of actual exponents** to be symmetric (roughly equal number of negatives and positives),
 - we need the **middle** of the exponent range to be 0.

n=8 (Single Precision)

- Total possible values: $2^8=256$
- Stored exponents: 0 to 255
- Choose **bias = 127**
- **actual exponent = stored exponent - 127**

Stored Exponent	Actual Exponent
0	-127
127	0
255	+128

- So, actual exponent range is: **-127 to +128**
- Bias shifts the range **so that 0 is in the center** of the stored values.

Final thoughts- Bias

- The bias centers the exponent range **around zero** so that:
 - Half the values represent **negative exponents**
 - Half represent **positive exponents**
- This way, we can **store both positive and negative powers of 2** using only **unsigned binary**.
- Formula:
 - n is the number of bit in exponent

$$\text{Bias} = 2^{n-1} - 1$$

Final thoughts-Bias

Bits in Exponent	Bias	Used In
5	15	Half precision
8	127	Single precision
11	1023	Double precision
15	16383	Quad precision

- **Bias** makes encoding/decoding of exponents symmetric and hardware-efficient, without needing negative binary storage.

Example: 8-bit Exponent (Bias = 127)

Actual Exponent	Stored Exponent	Binary (Stored)
-2	125	01111101
0	127	01111111
+2	129	10000001

- All exponents are stored as **positive values (0 to 255)** - **unsigned integers**.

Hardware Efficiency

1. No Signed Arithmetic

- Hardware just stores and reads unsigned binary.
- No need for two's complement or sign extension logic.

2. Simpler Comparison

- Comparing two exponents becomes just an **unsigned integer comparison**.
- Example: Is exponent A > exponent B? → just compare 8-bit numbers.

3. Symmetry

- The exponent range is **centered around 0**:
 - Bias = $2^{n-1}-1$
 - Ensures nearly equal range for negative and positive actual exponents.

Normalization

- **Normalization in floating-point representation refers to adjusting a number so that:**
 - The **most significant (leftmost) digit** before the binary point is **always 1** (for non-zero numbers).
- This ensures a **standardized form** of representing all numbers.
- It is essential for:
 - **Consistency**
 - **Maximum precision**
 - **Efficient hardware implementation**

Significance of Normalization

- Store $(6.25)_{10} = (110.01)_2$

- **WITHOUT NORMALIZATION:**

- It can be also store like this

- 0.11001×2^3
 - 1.011001×2^2

- Disadvantage

- **Waste bits:-** storing leading zeros (0.).
 - **Cannot assume** a consistent leading digit.
 - Hardware must handle many forms of the same number, which adds **complexity**.
 - Precision is **lost** because fewer bits are available for meaningful digits.

With Normalization: IEEE 754

- IEEE 754 is a **normalized** form.
 - **Precision:** You don't waste bits storing leading zeros or repeating patterns.
 - **Uniqueness:** A number has **only one** normalized form.
 - **Simplifies hardware:** Same format for all numbers makes arithmetic operations faster and more consistent.
- With normalized values,
- hardware knows every number looks like:
$$(-1)^S \times 1.M \times 2^{E-Bias}$$
- The **1.** is **implied**, not stored.
- No inconsistency- only one representation
- Precision is more

Double Precision

- **Double Precision?**
- Double precision uses **64 bits** to represent a floating-point number.
- Follows the **IEEE 754 standard**.

1 bit	11 bit	52 bit
Sign Bit	Biased Exponent	Significand/Mantissa

IEEE 754 Double-Precision Representation of 6.75			Example
Step	Description	Result	
Decimal Number	The number to be converted	6.75	
Binary Conversion	Convert integer and fraction separately: $6 \rightarrow 110$, $0.75 \rightarrow .11$	110.11	
Normalized Form	Convert to form $1.M \times 2^E$	1.1011×2^2	
Sign Bit (S)	0 for positive, 1 for negative	0	
Exponent (E)	Exponent in binary	2	
Bias	For double-precision, bias = 1023	1023	
Stored Exponent	$E + \text{Bias} = 2 + 1023 = 1025$	10000000001 (11 bits)	
Mantissa (M)	Remove leading 1., store remaining bits	1011000000000000000000000000000000 0000000000000000 (52 bits)	
Final 64-bit Format	IEEE 754 representation combining all parts	0 10000000001 1011000000000000000000000000000000 0000000000000000	

IEEE 754 Double-Precision Representation of 6.75			Example
Step	Description	Result	
Decimal Number	The number to be converted	6.75	
Binary Conversion	Convert integer and fraction separately: 6 → 110, 0.75 → .11	110.11	
Normalized Form	Convert to form 1.M × 2^E	1.1011 × 2 ²	
Sign Bit (S)	0 for positive, 1 for negative	0	
Exponent (E)	Exponent in binary	2	
Bias	For double-precision, bias = 1023	1023	
Stored Exponent	E + Bias = 2 + 1023 = 1025	10000000001 (11 bits)	
Mantissa (M)	Remove leading 1., store remaining bits	101100000000000000000000000000000000 0000000000000000 (52 bits)	
Final 64-bit Format	IEEE 754 representation combining all parts	0 10000000001 101100000000000000000000000000000000 0000000000000000	

[illegible]

IEEE 754 Double-Precision Representation of -6.5

Step	Description	Value
Decimal Number	The original number	-6.5
Binary Representation	Integer part: $6 \rightarrow 110$ Fractional part: $0.5 \rightarrow .1$	110.1
Normalized Binary	Shift to form $1.M \times 2^E$	1.101×2^2
Sign Bit (S)	1 for negative	1
Actual Exponent (E)	Power of 2 in normalized form	2
Bias	Bias for double-precision (11-bit exponent)	1023
Stored Exponent	$E + \text{Bias} = 2 + 1023$	1025
Binary Exponent Field	11-bit binary of 1025	1000000001
Mantissa (M)	Remove leading 1 from 1.101 $\rightarrow$ store 101000... up to 52 bits	10100000000000000000000000000000 00000000000000000000000000000000 00

Example: Decimal to IEEE 754 Single-Precision Format: **12.25**

Compute Sign , Exponent, and Mantissa

Field	Bit Width	Binary Value
Sign	1 bit	0
Exponent	8 bits	10000010
Mantissa	23 bits	10001000000000000000000

Decimal to IEEE 754 Single-Precision Format:

12.25

Step	Description	Value
1. Decimal Number	The number to convert	12.25
2. Binary Conversion	Integer part: 12 = 1100 Fraction part: 0.25 = 0.01	1100.01
3. Normalize Binary	Convert to form $1.M \times 2^E$	1.10001×2^3
4. Sign Bit (S)	0 for positive numbers	0
5. Actual Exponent (E)	From normalized form: exponent is 3	3
6. Bias	IEEE 754 single-precision uses a bias of 127	127
7. Stored Exponent	$E + \text{Bias} = 3 + 127 = 130$	130
8. Exponent Bits	Binary of 130 (8 bits)	10000010
9. Mantissa	Bits after the binary point in 1.10001, fill up to 23 bits	10001000000000000000000

- Example: Can the number 0.1 be stored exactly in single-precision. Justify your answer.

the binary representation of 0.1 is:

0.00011001100110011001100110011...

(repeats forever)

- IEEE 754 **single precision** has **only 23 bits** for the mantissa.
- It **truncates** or **rounds off** the repeating binary.
- That means **the stored value is an approximation**, not exact.

Overflow

- **Overflow** occurs when the **result of a floating-point operation** is **too large** (in magnitude) to be represented in the given format.
- The exponent **exceeds the maximum allowed value**
 - (e.g., > 255 for single-precision after bias).
- The result is replaced by **$+\infty$ or $-\infty$** , depending on the sign.
- **Example:**
 - Multiplying two very large numbers:
 - $1 \times 10^{38} \times 1 \times 10^{38} \Rightarrow +\infty$ (in single precision)

Positive overflow

- **Definition:**

- Positive overflow occurs when a **positive result exceeds the maximum representable finite value** in the floating-point format.

- **Causes:**

- Adding or multiplying two large positive numbers
- Exponent increases **beyond maximum** due to normalization or rounding

- **IEEE 754 Result:**

- Represented as **+Infinity**
- **Sign bit** = 0
- **Exponent** = all 1s (111...1)
- **Mantissa** = all 0s

Large_Value × Large_Value → +∞

Negative overflow

Definition:

- Negative overflow occurs when a **negative result becomes more negative than the minimum representable finite value.**

Causes:

- Adding two **large negative** numbers
- Multiplying a **large positive** with a large negative
- Exponent becomes **too large** in magnitude with **sign bit 1**

IEEE 754 Result:

- Represented as **−Infinity**
 - **Sign bit = 1**
 - **Exponent = all 1s (111...1)**
 - **Mantissa = all 0s**
- −Large_Value × Large_Value → $-\infty$**

Overflow Summary

Large_Value $\times$ Large_Value $\rightarrow +\infty$ **Positive overflow-** to large
-Large_Value $\times$ Large_Value $\rightarrow -\infty$ **Negative overflow-** to small

Condition	Cause	Result	Sign Bit	Exponent Field	Mantissa Field	Description
Positive Overflow	Result too large to represent	+Infinity	0	All 1s	All 0s	Value exceeds max finite number
Negative Overflow	Large negative result	-Infinity	1	All 1s	All 0s	Value below negative limit
NaN (Not a Number)	Invalid operation (0/0, $\infty - \infty$, $\sqrt{-x}$, etc.)	NaN	0 or 1	All 1s	$\neq$ All 0s	Undefined or unrepresentable result

Underflow

- **Underflow** occurs when the **result of a floating-point operation** is **too small** (in magnitude) to be represented as a **normalized** number.
- The exponent becomes **less than the minimum allowed value**
- For example: < 1 after bias for single-precision.
- The result may:
 - Be stored as a **denormalized number** (with reduced precision), or
 - Be rounded to **zero**, if too small even for **denormals**.
- **Example:**
 - Subtracting two very close small numbers:
 - $1.000001 \times 10^{-45} - 1.000000 \times 10^{-45} \Rightarrow \text{rounded to } 0$

Denormalized Number (Denorm) in IEEE 754?

- A **denormalized number (or denorm)**
 - It is a special kind of floating-point number.
 - Used in **IEEE 754** format.
 - Represents **very small values**.
 - **Which are too close to zero** to be stored as **normalized** numbers.

Why are denormals needed?

- **Normalized format** requires the form:
 $1.M \times 2^E$
- When the **exponent becomes too small**,
 - this form cannot hold the number.
 - Instead of underflowing to **zero**, IEEE 754 allows using **denormals** to preserve some precision:

$$0.M \times 2^{-126} (\text{in single-precision})$$

How Denormals Work (Single-Precision Example)

Field	Value for Denorm
Sign	0 (positive) or 1 (negative)
Exponent	All 0s: 00000000
Mantissa	Non-zero (e.g., 000...01)

The **leading 1** is **not assumed** (unlike normalized numbers).

Value becomes:

$$(-1)^{\text{sign}} \times 0.\text{Mantissa} \times 2^{-126}$$

Example

Let's bits are:

- Sign = 0
- Exponent = 00000000
- Mantissa = 00000000000000000000000001

Then the value is:

$+0.00000000000000000000000000000001 \times 2^{-126} \approx 1.4 \times 10^{-45}$

Positive underflow

- **Definition:**

- Positive underflow occurs when a **small positive number** becomes **too close to zero** to be represented in **normalized IEEE 754 format**.

- The **exponent** becomes

- **smaller than the minimum allowed** for normalized numbers
- (i.e., less than 1 after bias).

- If the value is **still representable**,

- it is stored as a **denormalized (subnormal)** number.

- If it's **too small even for denormals**, it is **rounded to +0**.

Small_Pos × Small_Pos → +0 or +Denorm

Negative underflow

- **Definition:**

- Negative underflow occurs when a **small negative number** becomes **too close to zero** to be represented as **a normalized** floating-point number.
- The exponent falls below the minimum representable range.
- If slightly above 0 in magnitude,
 - it's stored as a **negative denormal**.
- If smaller still, it's **rounded to -0**.

-Small × Small → -0 or -Denorm

Underflow Summary

Condition	Cause	Result	Sign Bit	Exponent Field	Mantissa Field	Description
Denormalize d Number	Exponent below normal range	Very small number	0 or 1	All 0s	≠ All 0s	Too small to normalize, precision reduced
Underflow to Zero	Smaller than smallest denormalize d number	Zero (+0 or -0)	0 or 1	All 0s	All 0s	Rounded down to absolute zero

IEEE 754 Floating Point Representation: Formula Table

Aspect	Single Precision (32-bit)	Double Precision (64-bit)
Total Bits	32 bits	64 bits
Sign Bit	1 bit	1 bit
Exponent Bits	8 bits	11 bits
Mantissa Bits	23 bits	52 bits
Exponent Bias	127	1023
Normalized Formula	$(-1)^S \times (1.M) \times 2^{(E-127)}$	$(-1)^S \times (1.M) \times 2^{(E-1023)}$
Denormalized Formula	$(-1)^S \times (0.M) \times 2^{-126}$	$(-1)^S \times (0.M) \times 2^{-1022}$
Zero	Exponent = 0, Mantissa = 0 $\rightarrow$ +0 or -0	Same
Infinity	Exponent = 255, Mantissa = 0 $\rightarrow$ $+\infty$ or $-\infty$	Exponent = 2047, Mantissa = 0 $\rightarrow$ $+\infty$ or $-\infty$
NaN (Not a Number)	Exponent = 255, Mantissa $\neq$ 0	Exponent = 2047, Mantissa $\neq$ 0

Floating Point Addition

Step 1: Extract Fields

- Extract **sign (S)**, **exponent (E)**, and **mantissa (M)** from both numbers A and B
- Assume hidden bit: mantissa becomes **1.M** (for normalized numbers)

Step 2: Align Exponents

- Calculate exponent difference: $d = |E_1 - E_2|$
- Shift the **mantissa of the smaller exponent** to the right by d bits
- Set both exponents to the **larger exponent**

Step 3: Add/Subtract Mantissas

- If **signs are the same**, **add** the mantissas
- If **signs are different**, **subtract** the smaller mantissa from the larger one
- Record the sign of the larger absolute value

Step 4: Normalize Result

- If the result is not in normalized form (1.M), shift mantissa:
 - **Left** if result $< 1 \rightarrow$ decrement exponent
 - **Right** if result $\geq 2 \rightarrow$ increment exponent

Step 5: Round the Result

- Round the mantissa to fit within 23 bits (single) or 52 bits (double)
- IEEE 754 uses **round to nearest even** as the default

Step 6: Handle Special Cases

- If exponent overflows $\rightarrow$ **Infinity**
- If exponent underflows $\rightarrow$ **Denormalized number or zero**
- If mantissa becomes zero $\rightarrow$ **Zero**

Step 7: Reconstruct IEEE 754 Result

Result = [Sign][Exponent + Bias][Mantissa without leading 1]

Pseudocode for Addition

1. function float_add(A, B):

2. Extract S1, E1, M1 from A
3. Extract S2, E2, M2 from B
4. Add hidden bit: $M1 = 1.M1$, $M2 = 1.M2$
5. if $E1 > E2$:
 6. shift M2 right by $(E1 - E2)$
 7. $E = E1$
8. else:
 9. shift M1 right by $(E2 - E1)$
 10. $E = E2$
11. if $S1 == S2$:
 12. $M_result = M1 + M2$
 13. $S_result = S1$
14. else:
 15. if $M1 > M2$:
 16. $M_result = M1 - M2$
 17. $S_result = S1$
 18. else:
 19. $M_result = M2 - M1$
 20. $S_result = S2$
21. Normalize M_result (shift left or right, adjust E)
22. Round M_result
23. Handle overflow/underflow/zero
24. Assemble final IEEE 754 result from S_result, E, M_result
25. return result

Floating point Addition

- Add 12.5 and 3.75
- floating-point addition using IEEE 754 (32-bit single-precision format)
- Solution:
- **Step 1: Convert to IEEE 754 Binary (Single Precision)**

Step 1: Extract Fields

- Extract **sign (S)**, **exponent (E)**, and **mantissa (M)** from both numbers A and B
- Assume hidden bit: mantissa becomes **1.M** (for normalized numbers)

Number	Binary	Normalized Form	Sign	Exponent (Bias 127)	Mantissa (23 bits)
12.5	1100.1	1.1001×2^3	0	$127 + 3 = 130 = 10000010$	1001000000000 0000000000
3.75	11.11	1.111×2^1	0	$127 + 1 = 128 = 10000000$	1110000000000 0000000000

Step 2: Align Exponents

- Calculate exponent difference: $d = |E_1 - E_2|$
- Shift the **mantissa of the smaller exponent** to the right by d bits
- Set both exponents to the **larger exponent**

- We must **align the smaller exponent** (here 128) to the larger one (130). To do this, shift the mantissa of the smaller exponent (3.75) **right** by $(130 - 128) = 2$ bits.

- Original 3.75 Mantissa: 1.111000000000000000000000
- Shifted Right by 2 bits: 0.011110000000000000000000

Step 3: Add/Subtract Mantissas

- If **signs are the same**, **add** the mantissas
- If **signs are different**, **subtract** the smaller mantissa from the larger one

- Record the sign of the larger absolute value

Now both numbers are in the form:

- 12.5: 1.100100000000000000000000
- 3.75 shifted: 0.011110000000000000000000

Add:

$$\begin{array}{r} 1.100100000000000000000000 \\ + 0.011110000000000000000000 \\ \hline 10.000010000000000000000000 \end{array}$$

Step 4: Normalize the Result

- The result 10.000010... is **not normalized** (two bits before the decimal point).
So we shift right once: 1.000001000000000000000000
Increment exponent by 1 $\rightarrow 130 + 1 = 131$ (10000011)

Step 5: Final IEEE 754 Format

Field	Value
Sign	0
Exponent	131 = 10000011
Mantissa	000001000000000000000000

Final value: 0 10000011 000001000000000000000000

ADD 2.5 and -1.5 In IEEE754 Single precision

Step1 : IEEE 754 REPRESENTATION

Step	Description	2.5	-1.5
1	Decimal	2.5	-1.5
2	Sign bit (S)	0	1
3	Binary	10.1	1.1
4	Normalized	1.01×2^1	1.1×2^0
5	Exponent (E)	1	0
6	Biased Exponent (E + 127)	128 $\rightarrow$ 10000000	127 $\rightarrow$ 01111111
7	Mantissa (23 bits)	01000000000000000000000 000	10000000000000000000000 000
8	IEEE 754 Representation	0 10000000 01000000000000000000000 000	1 01111111 10000000000000000000000 000

Step 2: Align Exponents (Shift smaller one)

Operation	Aligned Values
Exponent Difference	$128 - 127 = 1$
Shift -1.5's mantissa	Shift right by 1: $1.100... \rightarrow 0.110...$
Resulting Mantissas	2.5: 1.010000000000000000000000 -1.5: 0.110000000000000000000000

Step 3: Add/Subtract Mantissas (Signs differ → subtract)

Operation	Result
$1.010 - 0.110$	0.100
Decimal equivalent	1.0

Step 4: Normalize the Result

Binary Result	Normalized	Exponent Adjusted	New Biased Exponent
0.100	1.000×2^0	128	10000000

Step 5: Final IEEE 754 Result

Sign	Exponent (8 bits)	Mantissa (23 bits)	Final IEEE 754
0	10000000	00000000000000000000000 00	0 10000000 00000000000000000000000 00